

Tutorial: Conjuntos (Sets) em Python

Um guia introdutório com criação, operações e exemplos práticos.

1. O que são conjuntos?

Em Python, um conjunto (set) é uma coleção de elementos **sem duplicatas**. Os conjuntos são úteis para operações matemáticas como união, intersecção e diferença. Eles também são muito usados para verificar se um elemento pertence a uma coleção.

2. Como criar um conjunto

Use chaves `{ }` com elementos separados por vírgulas. Para criar um conjunto vazio, use `set()`, pois `{ }` cria um dicionário vazio.

```
frutas = {"maçã", "banana", "laranja"}
print(frutas)
```

```
conjunto_vazio = set()
print(conjunto_vazio)
```

3. Adicionando e removendo elementos

O método `add()` adiciona um elemento. O método `remove()` remove um elemento e gera erro se ele não existir. Já `discard()` remove o elemento, mas não gera erro caso ele não esteja no conjunto.

```
frutas.add("uva")
frutas.remove("banana")
frutas.discard("abacaxi")
```

4. União

A união reúne todos os elementos de dois conjuntos, eliminando duplicatas. Pode ser feita com o operador `|` ou com o método `union()`.

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
print(A | B)
print(A.union(B))
```

5. Intersecção

A intersecção retorna somente os elementos que aparecem nos dois conjuntos. Pode ser feita com `&` ou com `intersection()`.

```
print(A & B)
print(A.intersection(B))
```

6. Diferença

A diferença **A - B** retorna os elementos que estão em A, mas não estão em B. A ordem importa: **A - B** pode ser diferente de **B - A**.

```
print(A - B)
print(B - A)
```

7. Diferença simétrica

A diferença simétrica retorna os elementos que estão em apenas um dos conjuntos, mas não nos dois. Em Python, usamos $\wedge$ ou `symmetric_difference()`.

```
print(A ^ B)
print(A.symmetric_difference(B))
```

8. Pertencimento

Use **in** para verificar se um elemento pertence a um conjunto e **not in** para verificar o contrário.

```
print(2 in A)
print(10 in A)
```

9. Subconjuntos e superconjuntos

Um conjunto é subconjunto de outro quando todos os seus elementos estão contidos no outro conjunto. Use `issubset()` e `issuperset()` para essas verificações.

```
C = {1, 2}
print(C.issubset(A))
print(A.issuperset(C))
```

10. Tamanho do conjunto

A função `len()` informa quantos elementos existem no conjunto.

```
print(len(A))
```

Resumo das principais operações

A | B → união

A & B → intersecção

A - B → diferença

A ^ B → diferença simétrica

x in A → verifica pertencimento

len(A) → quantidade de elementos

Arquivo complementar: `exemplos_conjuntos.py`